

## Souvenirer

Har harr-y (namnet av huvudpersonen i storyn) köper souvenirs i en mystisk butik. I denna finns det  $N$  **sorters** souvenirs. Det finns oändligt många souvenirs av varje sort i butiken.

Varje sorts souvenir har ett visst pris. Mer exakt, en souvenir av typ  $i$  ( $0 \leq i < N$ ) kostar  $P[i]$  riksdaler, där  $P[i]$  är ett positivt heltal.

Har harr-y vet att souvenir-sorterna är sorterade i minskande ordning på kostnad, och att alla kostnader är unika. Formellt,  $P[0] > P[1] > \dots > P[N-1] > 0$ . Dessutom vet han värdet av  $P[0]$ . Tyvärr vet Har harr-y inget om priset av resten av souvenirerna.

För att köpa souvenirs kommer Har harr-y utföra ett flertal transaktioner med butiksbiträdet.

Varje transaktion består av följande steg:

1. Har harr-y ger butiksbiträdet ett visst (positivt) antal riksdaler till butiksbiträdet.
2. Butiksbiträdet lägger pengarna i ett hemligt fack, där Har harr-y inte kan se de.
3. Butiksbiträdet betraktar varje sorts souvenir av sort  $0, 1, \dots, N-1$  i den ordningen, en efter en. Varje sort betraktas **exakt en gång** per transaktion.
  - När butiksbiträdet betraktar souvenirerna av sort  $i$ , om antalet riksdaler i facket är minst  $P[i]$ , då händer följande:
    - butiksbiträdet tar bort  $P[i]$  riksdaler från facket, och
    - butiksbiträdet lägger in en souvenir av sort  $i$  i facket.
4. Butiksbiträdet tar alla souvenirs och kvarvarande riksdaler i facket och ger de till Har harr-y.

Notera att facket är helt tomt innan varje transaktion påbörjas; det innehåller varken riksdaler eller souvenirs.

Ditt uppdrag är att hjälpa Har harr-y att utföra ett flertal transaktioner, så att:

- i varje transaktion köper han **åtminstone en** souvenir, och
- sammanlagt köper han **exakt**  $i$  souvenirs av sort  $i$ , för varje  $i$  där  $0 \leq i < N$ . I synnerhet innebär detta att Har harr-y inte köper någon souvenir av sort 0.

Har harr-y behöver inte minimera antalet transaktioner och har ett oändligt antal riksdaler (han köpte gamestop).

# Implementationsdetaljer

Har harr-y hoppas du har det i dig att implementera följande procedur:

```
void buy_souvenirs(int N, long long P0)
```

- $N$ : antalet sorter souvenirer.
- $P0$ : värdet av  $P[0]$ .
- Denna proceduren kommer att anropas exakt en gång för varje testfall.

Den ovanstående proceduren kan anropa följande procedur för att säga åt Har harr-y att utföra en transaktion:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- $M$ : antalet riksdaler som Har harr-y ger till butiksbiträdet.
- Denna proceduren returnerar ett par. Det första elementet av paret är en lista  $L$ , som innehåller sorterna av souvenirer som köptes (i ökande ordning). Det andra elementet i paret är ett heltal  $R$ , antalet riksdaler som Har harr-y får tillbaka efter transaktionen.
- Det krävs att  $P[0] > M \geq P[N - 1]$ . Kravet  $P[0] > M$  säkerställer att Har harr-y inte köper en souvenir av sort 0, och  $M \geq P[N - 1]$  säkerställer att Har harr-y köper åtminstone en souvenir. Om dessa kräven inte uppfylls, så kommer din lösning få felmeddelandet `Output isn't correct: Invalid argument`. Notera att till skillnad från  $P[0]$ , så tillhandahålls inte värdet av  $P[N - 1]$  i indatan.
- Proceduren får anropas som mest 5000 gånger per testfall.

Domaren är **inte adaptiv**. Detta innebär att alla priserna  $P$  är fixerade innan `buy_souvenirs` anropas för första gånger.

## Begränsningar

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$  för varje  $i$  där  $0 \leq i < N$ .
- $P[i] > P[i + 1]$  för varje  $i$  där  $0 \leq i < N - 1$ .

## Subtasks

| Subtask | Poäng | Ytterligare begränsningar                                                                                                                     |
|---------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1       | 4     | $N = 2$                                                                                                                                       |
| 2       | 3     | $P[i] = N - i$ för varje $i$ där $0 \leq i < N$ .                                                                                             |
| 3       | 14    | $P[i] \leq P[i + 1] + 2$ för varje $i$ där $0 \leq i < N$ .                                                                                   |
| 4       | 18    | $N = 3$                                                                                                                                       |
| 5       | 28    | $P[i + 1] + P[i + 2] \leq P[i]$ för varje $i$ där $0 \leq i < N - 2$ .<br>$P[i] \leq 2 \cdot P[i + 1]$ för varje $i$ där $0 \leq i < N - 1$ . |
| 6       | 33    | Inga ytterligare begränsningar.                                                                                                               |

## Exempel

Betrakta följande anrop.

```
buy_souvenirs(3, 4)
```

Det finns  $N = 3$  sorters souvenirer och  $P[0] = 4$ . Notera att det bara finns tre möjliga sekvenser av kostnader  $P$ :  $[4, 3, 2]$ ,  $[4, 3, 1]$ , och  $[4, 2, 1]$ .

Anta att `buy_souvenirs` anropar `transaction(2)`, och låt oss säga att anropet returnerar  $([2], 1)$ , vilket innebär att Har harr-y köpte en souvenir av sort 2 och butiksbiträdet gav tillbaka 1 riksdal till honom. Notera att detta låter oss dra slutsatsen att  $P = [4, 3, 1]$ , eftersom:

- Om  $P = [4, 3, 2]$ , hade `transaction(2)` returnerat  $([2], 0)$ .
- Om  $P = [4, 2, 1]$ , hade `transaction(2)` returnerat  $([1], 0)$ .

Sedan kan `buy_souvenirs` anropa `transaction(3)`, som returnerar  $([1], 0)$ , vilket innebär att Har harr-y har köpt en souvenir av sort 1 och butiksbiträdet gav tillbaka 0 riksdaler. Hittills har han alltså sammanlagt köpt en souvenir av sort 1 och en av sort 2.

Till sist kan `buy_souvenirs` anropa `transaction(1)`, som då returnerar  $([2], 0)$ , vilket innebär att Har harr-y köpte en souvenir av sort 2. Notera att `transaction(2)` också hade funkat. Sammanlagt har Har harr-y nu köpt en souvenir av sort 1 och två av sort 2, och instansen är därmed löst.

## Exempeldomare

Indataformat:

$N$

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Utdataformat:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

$Q[i]$  är antalet souvenirer av sort  $i$  som programmet köpt av varje sorts souvenir för varje  $i$  där  $0 \leq i < N$ .